

## Regular Expressions write-up

Learn and practise using regular expressions.

### Task 1 Introduction

#### What are regular expressions?

Regular expressions (or Regex) are patterns of text that you define to search documents and match exactly what you're looking for.

#### Why should I learn how to use them?

Even if you won't need them sooner or later, it's a great tool to know how to use. It will make you more capable in CTF's, and potentially a better developer if that's a goal you have. You spend a little time learning it and save yourself lots of time in the long run by using it.

#### I know all that, but I'm lazy.

This is a lazy person's tutorial. There's a little reading, and then you learn by doing.

#### Where's the 'Deploy' button?

There's no machine to deploy. There are two ways to test your expressions. Either:

create a text file with some test paragraphs (in a Unix machine) and then use **egrep** **<pattern> <file>** to see what matches and what doesn't, or use an online editor like <https://regexr.com/>. You can add your own text in the "Text" field, and then type your expressions (patterns) in the "Expression" field. I recommend the second way.

### Task 2 Charsets

When searching for a specific string in a file or block of text, you can search for it as is, with `grep 'string' <file>`. But what happens if you want to search for patterns of text? For example, you could be looking for a word that starts with a specific letter, or any words that end with numbers. That's where Regular Expressions come in.

Both of the aforementioned problems can be solved by using charsets. A charset is defined by enclosing in [ square brackets ] the character(s), or range of characters that you want to match. Then, it finds every occurrence of the pattern you have defined in the file/text you are searching.

`[abc]` will match a, b, and c (every occurrence of each letter)

`[abc]zz` will match azz, bzz, and czz.

You can also use a - dash to define ranges:

`[a-c]zz` is the same as above.

And then you can combine ranges together:

`[a-cx-z]zz` will match azz, bzz, czz, xzz, yzz, and zzz.

Most notably, this can be used to match any alphabetical character:

`[a-zA-Z]` will match any single letter (lowercase or uppercase).

You can use numbers too:

`file[1-3]` will match `file1`, `file2`, and `file3`.

Then, there is a way to exclude characters from a charset with the `^` hat symbol, and include everything else.

`[^k]ing` will match `ring`, `sing`, `$ing`, but not `king`.

Of course, you can exclude charsets, not just single characters.

`[^a-c]at` will match `fat` and `hat`, but not `bat` or `cat`.

Note 1: Don't confuse strings with charsets. The charset `[abc]` will match the string `abc`, but also `cba` and `ca`. It doesn't match the string, but rather every occurrence of the specified characters in that string.

Note 2: When specifying charsets, you should type the letters in the same order they appear in the questions, to avoid typing something correct that is not the right answer.

Note 3: Answering some of these questions is going to be tricky. Often times there are many different patterns that match specific strings. That means (as stated in the previous note) that you may find a proper solution that isn't the right answer for this room (because there can only be one). The right answer is typically the most efficient regex for that question.

**Efficient in this context means 2 things:**

1. **Be specific.** Here's an example: you could match any character from `a` to `c` using the `[a-z]` charset. But if the question only requires you to match characters from `a` to `c`, you should use the `[a-c]` charset, not `[a-z]`.

2. **Don't be too specific.** In contrast to the previous example, if a question requires you to match `a`, `c`, `f`, `r`, `s`, `z`, at that point, the expression that matches those specific characters would get longer and more complicated. So, it would make more sense to use `[a-z]`, because it is short and simple.

To reiterate, there cannot be one single correct solution. So if you've tested your solution and it works, you can take a break and come back to it later, or ask for a hint in discord, but try not to get frustrated.

Answer the questions below

Match all of the following characters: `c`, `o`, `g`

**Answer:** `[cog]`

Match all of the following words: `cat`, `fat`, `hat`

**Answer:** `[cfh]at`

Match all of the following words: `Cat`, `cat`, `Hat`, `hat`

**Answer:** `[CcHh]at`

Match all of the following filenames: `File1`, `File2`, `file3`, `file4`, `file5`, `File7`, `file9`

**Answer:** `[Ff]ile[1-9]`

Match all of the filenames of question 4, except `"File7"` (use the hat symbol)

**Answer:** `[Ff]ile[^7]`

### Task 3 Wildcards and optional characters

The wildcard that is used to match any single character (except the line break) is the `.` dot. That means that `a.c` will match `aac`, `abc`, `a0c`, `a!c`, and so on.

Also, you can set a character as optional in your pattern using the `?` question mark. That means that `abc?` will match `ab` and `abc`, since the `c` is optional.

Note: If you want to search for `.` a literal dot, you have to escape it with a `\` reverse slash. That means that `a.c` will match `a.c`, but also `abc`, `a@c`, and so on. But `a\.c` will match just `a.c`.

### Answer the questions below

Match all of the following words: Cat, fat, hat, rat

**Answer:** `.at`

Match all of the following words: Cat, cats

**Answer:** `[Cc]ats?`

Match the following domain name: cat.xyz

**Answer:** `cat\.xyz`

Match all of the following domain names: cat.xyz, cats.xyz, hats.xyz

**Answer:** `[ch]ats?\.xyz`

Match every 4-letter string that doesn't end in any letter from n to z

**Answer:** `...[^n-z]`

Match bat, bats, hat, hats, but not rat or rats (use the hat symbol)

**Answer:** `[^r]ats?`

### Task 4 Metacharacters and repetitions

There are easier ways to match bigger charsets. For example, `\d` is used to match any single digit. Here's a reference:

`\d` matches a digit, like 9

`\D` matches a non-digit, like A or @

`\w` matches an alphanumeric character, like a or 3

`\W` matches a non-alphanumeric character, like ! or #

`\s` matches a whitespace character (spaces, tabs, and line breaks)

`\S` matches everything else (alphanumeric characters and symbols)

Note: Underscores `_` are included in the `\w` metacharacter and not in `\W`. That means that `\w` will match every single character in `test_file`.

Often we want a pattern that matches many characters of a single type in a row, and we can do that with repetitions. For example, `{2}` is used to match the preceding character (or metacharacter, or charset) two times in a row. That means that `z{2}` will match exactly `zz`.

Here's a reference for each repetition along with how many times it matches the preceding pattern:

{12} - exactly 12 times.

{1,5} - 1 to 5 times.

{2,} - 2 or more times.

\* - 0 or more times.

+ - 1 or more times.

**Answer the questions below**

**Match the following word: catssss**

**Answer: `cats{4}`**

**Match all of the following words (use the \* sign): Cat, cats, catsss**

**Answer: `[Cc]ats*`**

**Match all of the following sentences (use the + sign): regex go br, regex go brrrrrr**

**Answer: `regex go br+`**

**Match all of the following filenames: ab0001, bb0000, abc1000, cba0110, c0000 (don't use a metacharacter)**

**Answer: `[abc]{1,3}[01]{4}`**

**Match all of the following filenames: File01, File2, file12, File20, File99**

**Answer: `[Ff]ile\d{1,2}`**

**Match all of the following folder names: kali tools, kali tools**

**Answer: `kali\s+tools`**

**Match all of the following filenames: notes~, stuff@, gtFOB#, lmaoo!**

**Answer: `\w{5}\W`**

**Match the string in quotes (use the \* sign and the \s, \S metacharacters):**

**"2f0h@f0j0%! a)K!F49h!FFOK"**

**Answer: `\S*\s*\S*`**

**Match every 9-character string (with letters, numbers, and symbols) that doesn't end in a "!" sign**

**Answer: `\S{8}[^!]`**

**Match all of these filenames (use the + symbol): .bash\_rc, .unnecessarily\_long\_filename, and note1**

**Answer: `\.?\w+`**

**Task 5 Starts with/ ends with, groups, and either/ or**

Sometimes it's very useful to specify that we want to search by a certain pattern in the beginning or the end of a line. We do that with these characters:

**^ - starts with**

**\$ - ends with**

So for example, if you want to search for a line that starts with abc, you can use ^abc.  
If you want to search for a line that ends with xyz, you can use xyz\$.

Note: The ^ hat symbol is used to exclude a charset when enclosed in [square brackets], but when it is not, it is used to specify the beginning of a word.

You can also define groups by enclosing a pattern in (parentheses). This function can be used for many ways that are not in the scope of this tutorial. We will use it to define an either/ or pattern, and also to repeat patterns. To say "or" in Regex, we use the | pipe.

For an "either/or" pattern example, the pattern during the (day|night) will match both of these sentences: during the day and during the night.

For a repetition example, the pattern (no){5} will match the sentence nonononono.

**Answer the questions below**

**Match every string that starts with "Password:" followed by any 10 characters excluding "0", irrespective of the position.**

**Answer:** `Password:[^0]{10}`

**Match "username: " in the beginning of a line (note the space!)**

**Answer:** `^username:\s`

**Match every line that doesn't start with a digit (use a metacharacter)**

**Answer:** `^!D`

**Match this string at the end of a line: EOF\$**

**Answer:** `EOF\$$`

**Match all of the following sentences:**

I use nano

I use vim

**Answer:** `I use (nano|vim)`

**Match all lines that start with \$, followed by any single digit, followed by \$, followed by one or more non-whitespace characters**

**Answer:** `\$\d\$\S+`

**Match every possible IPv4 IP address (use metacharacters and groups)**

**Answer:** `(\d{1,3}\.){3}\d{1,3}`

**Match all of these emails while also adding the username and the domain name (not the TLD) in separate groups (use \w): hello@tryhackme.com, username@domain.com, dummy\_email@xyz.com**

**Answer:** `(\w+)\@(\w+)\.com`

**Task 6 Conclusion**

Well done.

Regular expressions are very powerful, even at their most basic usage. There are many resources to study and practise online as well, which I strongly recommend.

Also, if you're planning on using regex to develop something and you want to search for something like an e-mail, you should search for premade expressions instead of writing your own.

With regex, you have to think specific, but not too specific, because then you might come up with complicated solutions when there are other more elegant and simple ones.

**Happy Hacking!!!**